Exercícios - Docker e Kubernetes (Entrega Final)

Identificação

Esta entrega reúne os 10 exercícios de Docker e Kubernetes, cada um com comandos, prints e respostas.

Campo	Informação
Aluno	Lucas
Disciplina	Projeto Interdisciplinar para Sistemas de Informação IV (confirmar)
Ambiente	Windows 11 Home, Docker Desktop, minikube v1.39.0 (driver docker)
Prefixo das imagens	lucas/ (no lugar de seu-nome/ do enunciado)

Parte 1: Docker (exercícios 1 a 4)

Os quatro exercícios mostram que uma imagem Docker é um molde e que cada container criado dela roda isolado.

Exercício 1: Rodando o primeiro container

Comandos:
```
docker run -it python:3.9-slim
```
Dentro do Python:
```
print("Olá, Docker!")
exit()
```
Em anexo
O que acontece quando você sai do container? Ele é encerrado, porque o processo principal (o interpretador Python) terminou.
Esse ambiente é persistente? Não. O que foi criado dentro do container se perde ao removê-lo, a menos que se use volumes.

Exercício 2: Construindo sua própria imagem

Alteração no app.py: a mensagem passou a ser "Minha primeira aplicação em container!".
Comandos:
```
docker build -t lucas/minha-app:v2 .
docker run -p 8081:8080 lucas/minha-app:v2
```
Em anexo

Qual a diferença entre imagem e container? A imagem é o molde, somente leitura, com o código e as dependências. O container é uma instância em execução criada a partir da imagem, e de uma imagem podem sair vários containers.
O que acontece se alterar o código sem rebuildar? Nada muda. O código foi copiado para dentro da imagem no momento do build, então o container continua com a versão

antiga até a imagem ser reconstruída.

Exercício 3: Trabalhando com múltiplos containers

Comandos:

```
docker run -d -p 8082:8080 --name app1 lucas/minha-app:v2
docker run -d -p 8083:8080 --name app2 lucas/minha-app:v2
docker ps
```

Em anexo

Os containers compartilham dados? Não. Cada container tem o seu próprio sistema de arquivos e é isolado dos demais.
Por que precisamos mapear portas diferentes? Duas aplicações não podem ocupar a mesma porta da máquina host. Dentro dos containers as duas escutam na 8080 sem conflito, e o mapeamento (8082 e 8083) as diferencia por fora.

Exercício 4: Alterando o comportamento da aplicação

Alteração no app.py: a mensagem passou a mostrar o nome e a disciplina: "Lucas - Projeto Interdisciplinar para Sistemas de Informação IV".
Comandos:

```
docker build -t lucas/minha-app:v3 .
docker run -d -p 8084:8080 --name app3 lucas/minha-app:v3
```

Em anexo

Parte 2: Kubernetes com Minikube (exercícios 5 a 9)

Os cinco exercícios levam a aplicação do Docker para um cluster Kubernetes local, passando por Deployment, escala, Service e debug.

Exercício 5: Subindo o Minikube

Comandos:

```
minikube start --driver=docker
kubectl get nodes
```

Em anexo

Quantos nós aparecem? Um nó, chamado minikube, com a função control-plane.
Qual o status deles? Ready.
Observação: o comando minikube docker-env | Invoke-Expression, indicado no guia, não funcionou, porque esta versão do minikube usa o runtime containerd (erro SSH_AGENT_START). No lugar dele, as imagens foram enviadas ao cluster com minikube image load:

```
docker build -t lucas/hello-app:v1 .
minikube image load lucas/hello-app:v1
```

Em anexo

Exercício 6: Criando um Deployment

Comandos:

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl delete pod NOME-DO-POD
kubectl get pods
```
Em anexo

Quantos pods estão rodando? Dois, conforme replicas: 2 no deployment.
O que acontece se você deletar um pod manualmente? O Deployment percebe que faltou uma réplica e cria um novo pod automaticamente, mantendo o número desejado.

Exercício 7: Escalando a aplicação

Alteração: no deployment.yaml, replicas: 2 passou para replicas: 4.
Comandos:
```
kubectl apply -f deployment.yaml
kubectl get pods
```
Em anexo

O que mudou? O número de pods passou de 2 para 4. Os pods existentes continuaram rodando e o Kubernetes criou só os que faltavam, sem derrubar a aplicação.
Isso seria útil em qual cenário real? Em picos de acesso, como Black Friday ou lançamento de produto. Mais réplicas dividem a carga e aumentam a disponibilidade, e depois do pico dá para reduzir e economizar recursos.

Exercício 8: Expondo a aplicação

Comandos:
```
kubectl apply -f service.yaml
kubectl get services
minikube service hello-app-service
```
Em anexo

Qual a função do Service? Dar um endereço estável à aplicação e distribuir o tráfego entre os pods.
O que aconteceria sem ele? Os pods teriam IPs que mudam a cada recriação, e a aplicação não teria acesso estável nem balanceamento de carga a partir de fora do cluster.

Exercício 9: Debug básico

Comandos:
```
kubectl get pods
kubectl describe pod NOME-DO-POD
kubectl logs NOME-DO-POD
```
Em anexo

Onde você vê erros? No describe, na seção Events (problemas de imagem, agendamento, falha ao iniciar), e nos logs, para erros da própria aplicação.
Qual comando é mais útil? O describe, quando o pod nem sobe; o logs, quando ele sobe mas a aplicação falha.

Parte 3: Customização completa (exercício 10)

O exercício 10 integra tudo: uma aplicação nova, com Dockerfile, imagem, deployment e service, rodando no Kubernetes e mostrando o nome do aluno e a data atual.

Código dos arquivos

app.py

```
from http.server import SimpleHTTPRequestHandler, HTTPServer
from datetime import datetime

class MyHandler(SimpleHTTPRequestHandler):
    def do_GET(self):
        agora = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
        msg = f"Aluno: Lucas - Data atual: {agora}"
        self.send_response(200)
        self.send_header("Content-type", "text/html; charset=utf-8")
        self.end_headers()
        self.wfile.write(msg.encode("utf-8"))

HTTPServer(('0.0.0.0', 8080), MyHandler).serve_forever()
```

Dockerfile

```
FROM python:3.9-slim
WORKDIR /app
COPY . .
EXPOSE 8080
CMD ["python", "app.py"]
```

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: data-app-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: data-app
  template:
    metadata:
      labels:
        app: data-app
    spec:
      containers:
      - name: data-app-container
        image: lucas/data-app:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 8080
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: data-app-service
spec:
```

```
  type: NodePort
  selector:
    app: data-app
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 8080
```

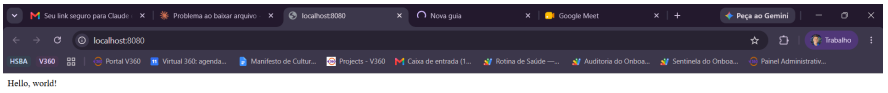
Execução

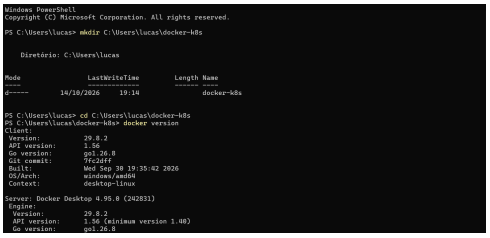
```
docker build -t lucas/data-app:v1 .
minikube image load lucas/data-app:v1
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
minikube service data-app-service
```

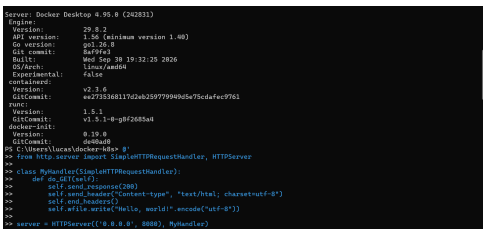
Em anexo

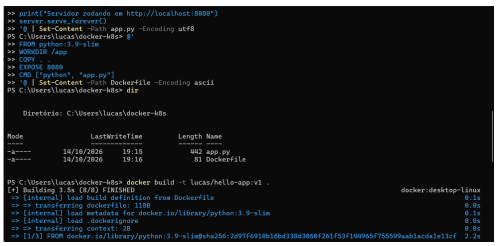
Conclusão

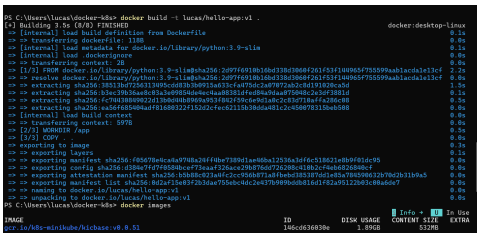
A atividade mostrou o caminho completo de uma aplicação: empacotada em uma imagem Docker, executada em containers isolados e depois orquestrada pelo Kubernetes, que recriou pods deletados, escalou de 2 para 4 réplicas e expôs a aplicação por um Service.

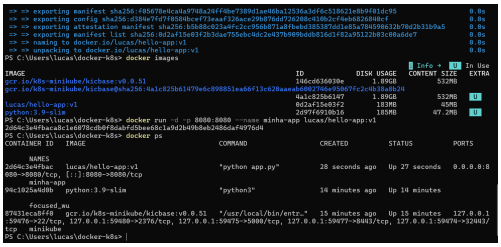










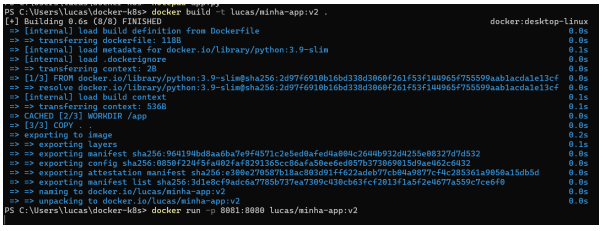


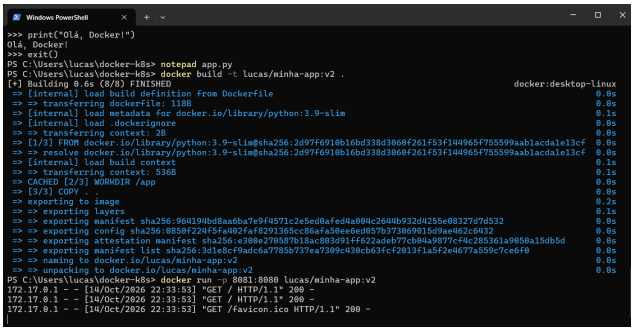

```
PS C:\Users\lucas\docker-k8s> docker run -it python:3.9-slim
Python 3.9.25 (main, Oct 31 2025, 23:16:49)
[GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Olá, Docker!")
Olá, Docker!
>>> exit()
PS C:\Users\lucas\docker-k8s>
```

```
from http.server import SimpleHTTPRequestHandler, HTTPServer

class MyHandler(SimpleHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/html; charset=utf-8")
        self.end_headers()
        self.wfile.write("Minha primeira aplicação em container!".encode("utf-8"))

server = HTTPServer(('0.0.0.0', 8080), MyHandler)
print("Servidor rodando em http://localhost:8080")
server.serve_forever()
```





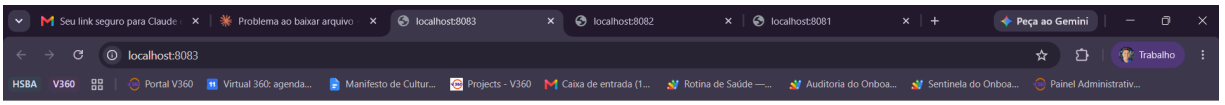
```
PS C:\Users\lucas\docker-k8s> docker run -p 8081:8080 lucas/minha-app:v2
172.17.0.1 - - [14/Oct/2026 22:33:53] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [14/Oct/2026 22:33:53] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [14/Oct/2026 22:33:53] "GET /favicon.ico HTTP/1.1" 200 -
Servidor rodando em http://localhost:8080
Traceback (most recent call last):
  File "/app/app.py", line 12, in <module>
    server.serve_forever()
  File "/usr/local/lib/python3.9/socketserver.py", line 232, in serve_forever
    ready = selector.select(poll_interval)
  File "/usr/local/lib/python3.9/selectors.py", line 416, in select
    fd_event_list = self._selector.poll(timeout)
KeyboardInterrupt
PS C:\Users\lucas\docker-k8s> notepad app.py
PS C:\Users\lucas\docker-k8s> docker run -p 8081:8080 lucas/minha-app:v2
172.17.0.1 - - [15/Oct/2026 02:48:41] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:41] "GET /favicon.ico HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:43] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:43] "GET /favicon.ico HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:44] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:44] "GET /favicon.ico HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:45] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:45] "GET /favicon.ico HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:50] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:50] "GET /favicon.ico HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:56] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:48:56] "GET /favicon.ico HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:49:14] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [15/Oct/2026 02:49:14] "GET /favicon.ico HTTP/1.1" 200 -
Servidor rodando em http://localhost:8080
```

Windows PowerShell

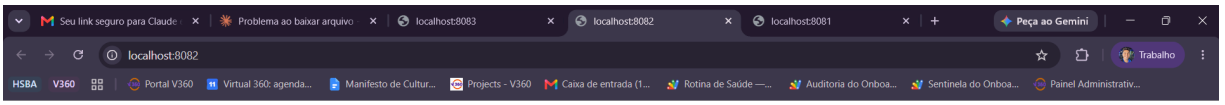
```
    server.serve_forever()
  File "/usr/local/lib/python3.9/socketserver.py", line 232, in serve_forever
    ready = selector.select(poll_interval)
  File "/usr/local/lib/python3.9/selectors.py", line 416, in select
    fd_event_list = self._selector.poll(timeout)
KeyboardInterrupt
PS C:\Users\lucas\docker-k8s> docker run -d -p 8082:8080 --name app1 lucas/minha-app:v2
689682bbcfb19d879f0a22267ddee4588bc7bd3d84be37ebacad3eade200fdc6
PS C:\Users\lucas\docker-k8s> docker run -d -p 8083:8080 --name app2 lucas/minha-app:v2
51df58240aa7168191513406de845a4cc25cca347b80bb10d9fc6448807ca393
PS C:\Users\lucas\docker-k8s> docker ps
CONTAINER ID   IMAGE                                 COMMAND                  CREATED         STATUS         PORTS

      NAMES
51df58240aa7   lucas/minha-app:v2                    "python app.py"          18 seconds ago   Up 17 seconds   0.0.0.0:8
083->8080/tcp, [::]:8083->8080/tcp
      app2
689682bbcfb1   lucas/minha-app:v2                    "python app.py"          18 seconds ago   Up 17 seconds   0.0.0.0:8
082->8080/tcp, [::]:8082->8080/tcp
      app1
2d64c3e4fbac   lucas/hello-app:v1                    "python app.py"          5 hours ago      Up 5 hours      0.0.0.0:8
080->8080/tcp, [::]:8080->8080/tcp
      minha-app
94c1025a4d0b   python:3.9-slim                       "python3"                5 hours ago      Up 5 hours

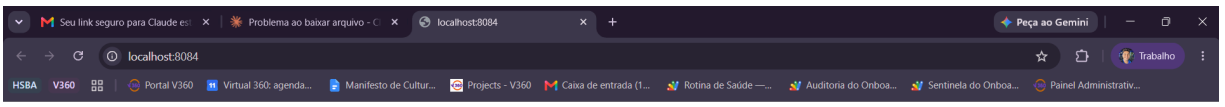
      focused_wu
87431eca8ff0   gcr.io/k8s-minikube/kicbase:v0.0.51   "/usr/local/bin/entr…"   5 hours ago      Up 5 hours      127.0.0.1
:59476->22/tcp, 127.0.0.1:59480->2376/tcp, 127.0.0.1:59475->5000/tcp, 127.0.0.1:59477->8443/tcp, 127.0.0.1:59474->32443/
tcp   minikube
PS C:\Users\lucas\docker-k8s>
```



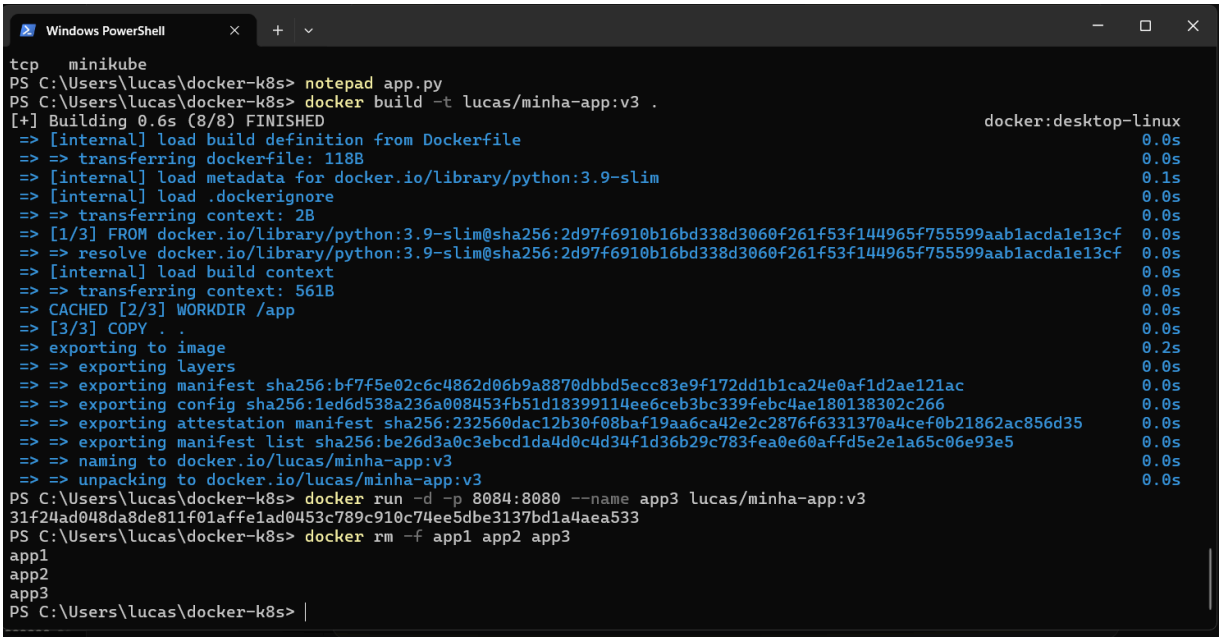
Minha primeira aplicação em container!



Minha primeira aplicação em container!



Lucas - Projeto Interdisciplinar para Sistemas de Informação IV




```
PS C:\Users\lucas\docker-k8s> minikube start --driver=docker
* minikube v1.39.0 em Microsoft Windows 11 Home Single Language 25H2
* Usando o driver docker com base no perfil existente
* Iniciando o nó primary control-plane "minikube" no cluster "minikube"
* Baixando a imagem base v0.0.51 ...
* Atualizando o docker "minikube" container em execução ...
! Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine
* Para baixar novas imagens externas, você pode precisar configurar um proxy: https://minikube.sigs.k8s.io/docs/referenc
e/networking/proxy/
* Preparando o Kubernetes v1.37.0 no containerd 2.3.4 ...
* Verificando os componentes do Kubernetes...
  - Usando a imagem gcr.io/k8s-minikube/storage-provisioner:v5
* Complementos habilitados: storage-provisioner, default-storageclass
* Concluído! O kubectl agora está configurado para usar o cluster "minikube" e o namespace "default" por padrão
PS C:\Users\lucas\docker-k8s> kubectl get nodes
NAME       STATUS   ROLES           AGE     VERSION
minikube   Ready    control-plane   4h55m   v1.37.0
PS C:\Users\lucas\docker-k8s> minikube docker-env | Invoke-Expression
! Usar o comando docker-env com o runtime containerd é um recurso altamente experimental, por favor, forneça feedback ou
 contribua para torná-lo melhor
X Saindo devido a SSH_AGENT_START: starting an SSH agent on Windows is not yet supported
```

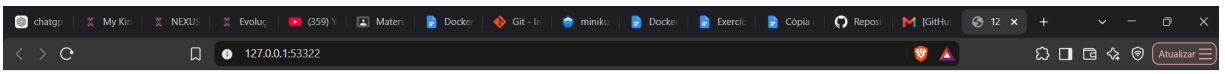
```
PS C:\Users\lucas\docker-k8s> docker build -t lucas/hello-app:v1 .
[+] Building 0.3s (8/8) FINISHED                                                                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                            0.0s
 => => transferring dockerfile: 118B                                                                                            0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                              0.1s
 => [internal] load .dockerignore                                                                                               0.0s
 => => transferring context: 2B                                                                                                 0.0s
 => [1/3] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf               0.0s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf               0.0s
 => [internal] load build context                                                                                               0.0s
 => => transferring context: 57B                                                                                                0.0s
 => CACHED [2/3] WORKDIR /app                                                                                                   0.0s
 => CACHED [3/3] COPY . .                                                                                                       0.0s
 => exporting to image                                                                                                          0.1s
 => => exporting layers                                                                                                         0.0s
 => => exporting manifest sha256:bf7f5e02c6c4862d06b9a8870dbbd5ecc83e9f172dd1b1ca24e0af1d2ae121ac                               0.0s
 => => exporting config sha256:1ed6d538a236a008453fb51d18399114ee6ceb3bc339febc4ae180138302c266                                 0.0s
 => => exporting attestation manifest sha256:4f29b81084e754910eef6fb8f408b8cfcfe72b0c5859c6e5e6ca93f41f06a9fe                   0.0s
 => => exporting manifest list sha256:cf64f4a4a38e3cafa183a86e1b53500eb204facbb34e3e603f1168ead30db75c                          0.0s
 => => naming to docker.io/lucas/hello-app:v1                                                                                   0.0s
 => => unpacking to docker.io/lucas/hello-app:v1                                                                                0.0s
PS C:\Users\lucas\docker-k8s> minikube image load lucas/hello-app:v1
```

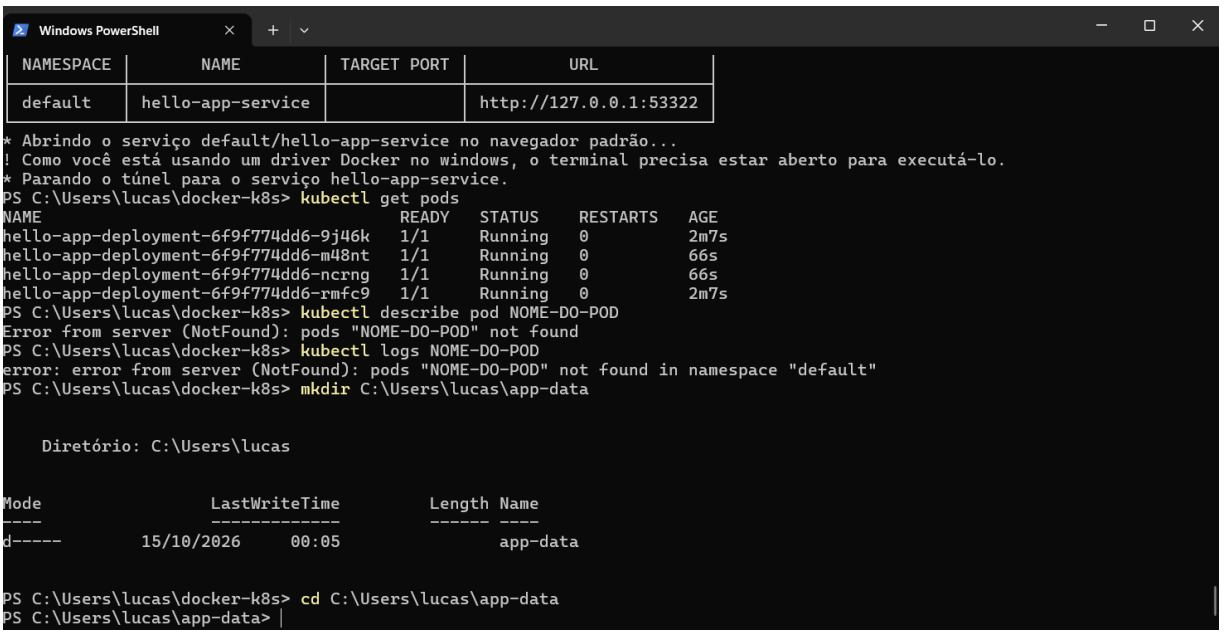
```
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf  0.0s
 => [internal] load build context                                                                                  0.0s
 => => transferring context: 57B                                                                                   0.0s
 => CACHED [2/3] WORKDIR /app                                                                                      0.0s
 => CACHED [3/3] COPY . .                                                                                          0.0s
 => exporting to image                                                                                             0.1s
 => => exporting layers                                                                                            0.0s
 => => exporting manifest sha256:bf7f5e02c6c4862d06b9a8870dbbd5ecc83e9f172dd1b1ca24e0af1d2ae121ac                  0.0s
 => => exporting config sha256:1ed6d538a236a008453fb51d18399114ee6ceb3bc339febc4ae180138302c266                    0.0s
 => => exporting attestation manifest sha256:4f29b81084e754910eef6fb8f408b8cfcfe72b0c5859c6e5e6ca93f41f06a9fe      0.0s
 => => exporting manifest list sha256:cf64f4a4a38e3cafa183a86e1b53500eb204facbb34e3e603f1168ead30db75c             0.0s
 => => naming to docker.io/lucas/hello-app:v1                                                                      0.0s
 => => unpacking to docker.io/lucas/hello-app:v1                                                                   0.0s
PS C:\Users\lucas\docker-k8s> minikube image load lucas/hello-app:v1
PS C:\Users\lucas\docker-k8s> minikube image ls
registry.k8s.io/pause:3.10.2
registry.k8s.io/kube-scheduler:v1.37.0
registry.k8s.io/kube-proxy:v1.37.0
registry.k8s.io/kube-controller-manager:v1.37.0
registry.k8s.io/kube-apiserver:v1.37.0
registry.k8s.io/etcd:3.7.0-0
registry.k8s.io/coredns/coredns:v1.14.6
gcr.io/k8s-minikube/storage-provisioner:v5
docker.io/lucas/hello-app:v1
docker.io/kindest/kindnetd:v20260820-69b56db7
docker.io/kindest/kindnetd:v20250512-df8de77b
```

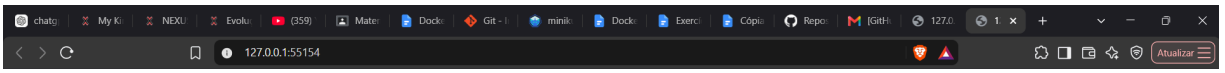
```
 => => unpacking to docker.io/lucas/hello-app:v1
PS C:\Users\lucas\docker-k8s> minikube image load lucas/hello-app:v1
PS C:\Users\lucas\docker-k8s> minikube image ls
registry.k8s.io/pause:3.10.2
registry.k8s.io/kube-scheduler:v1.37.0
registry.k8s.io/kube-proxy:v1.37.0
registry.k8s.io/kube-controller-manager:v1.37.0
registry.k8s.io/kube-apiserver:v1.37.0
registry.k8s.io/etcd:3.7.0-0
registry.k8s.io/coredns/coredns:v1.14.6
gcr.io/k8s-minikube/storage-provisioner:v5
docker.io/lucas/hello-app:v1
docker.io/kindest/kindnetd:v20260820-69b56db7
docker.io/kindest/kindnetd:v20250512-df8de77b
PS C:\Users\lucas\docker-k8s> kubectl apply -f deployment.yaml
deployment.apps/hello-app-deployment created
PS C:\Users\lucas\docker-k8s> kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
hello-app-deployment-6f9f774dd6-9j46k   1/1     Running   0          2s
hello-app-deployment-6f9f774dd6-rmfc9   1/1     Running   0          2s
PS C:\Users\lucas\docker-k8s> kubectl delete pod hello-app-deployment-7d9c8b6f5-abcde
Error from server (NotFound): pods "hello-app-deployment-7d9c8b6f5-abcde" not found
PS C:\Users\lucas\docker-k8s> kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
hello-app-deployment-6f9f774dd6-9j46k   1/1     Running   0          8s
hello-app-deployment-6f9f774dd6-rmfc9   1/1     Running   0          8s
PS C:\Users\lucas\docker-k8s>
```

```
PS C:\Users\lucas\docker-k8s> kubectl apply -f deployment.yaml
deployment.apps/hello-app-deployment created
PS C:\Users\lucas\docker-k8s> kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
hello-app-deployment-6f9f774dd6-9j46k   1/1     Running   0          2s
hello-app-deployment-6f9f774dd6-rmfc9   1/1     Running   0          2s
PS C:\Users\lucas\docker-k8s> kubectl delete pod hello-app-deployment-7d9c8b6f5-abcde
Error from server (NotFound): pods "hello-app-deployment-7d9c8b6f5-abcde" not found
PS C:\Users\lucas\docker-k8s> kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
hello-app-deployment-6f9f774dd6-9j46k   1/1     Running   0          8s
hello-app-deployment-6f9f774dd6-rmfc9   1/1     Running   0          8s
PS C:\Users\lucas\docker-k8s> notepad deployment.yaml
PS C:\Users\lucas\docker-k8s> kubectl apply -f deployment.yaml
deployment.apps/hello-app-deployment configured
PS C:\Users\lucas\docker-k8s> kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
hello-app-deployment-6f9f774dd6-9j46k   1/1     Running   0          61s
hello-app-deployment-6f9f774dd6-m48nt   1/1     Running   0          0s
hello-app-deployment-6f9f774dd6-ncrng   1/1     Running   0          0s
hello-app-deployment-6f9f774dd6-rmfc9   1/1     Running   0          61s
PS C:\Users\lucas\docker-k8s>
```

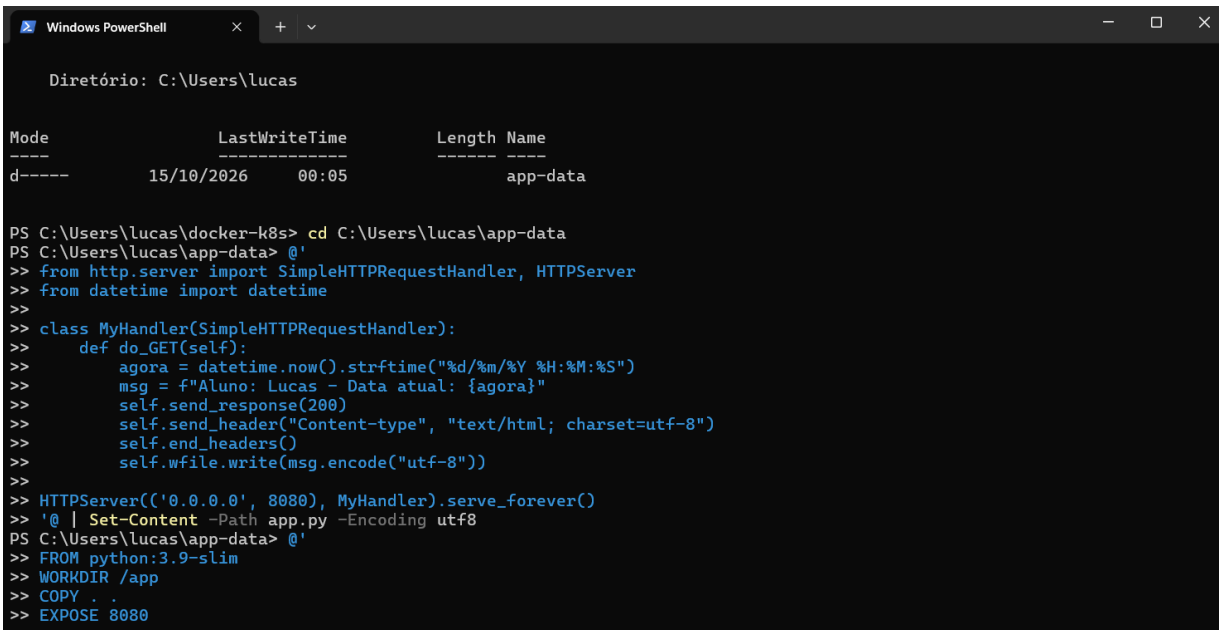
```
PS C:\Users\lucas\docker-k8s> notepad deployment.yaml
PS C:\Users\lucas\docker-k8s> kubectl apply -f deployment.yaml
deployment.apps/hello-app-deployment configured
PS C:\Users\lucas\docker-k8s> kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
hello-app-deployment-6f9f774dd6-9j46k   1/1     Running   0          61s
hello-app-deployment-6f9f774dd6-m48nt   1/1     Running   0          0s
hello-app-deployment-6f9f774dd6-ncrng   1/1     Running   0          0s
hello-app-deployment-6f9f774dd6-rmfc9   1/1     Running   0          61s
PS C:\Users\lucas\docker-k8s> kubectl apply -f service.yaml
service/hello-app-service created
PS C:\Users\lucas\docker-k8s> kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
hello-app-service   NodePort    10.103.79.193   <none>        8080:30139/TCP   1s
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          5h
PS C:\Users\lucas\docker-k8s> minikube service hello-app-service
|-----------|-------------------|-------------|---------------------------|
| NAMESPACE |       NAME        | TARGET PORT |            URL            |
|-----------|-------------------|-------------|---------------------------|
| default   | hello-app-service | 8080        | http://192.168.49.2:30139 |
|-----------|-------------------|-------------|---------------------------|
* Iniciando o túnel para o serviço hello-app-service.
|-----------|-------------------|-------------|------------------------|
| NAMESPACE |       NAME        | TARGET PORT |          URL           |
|-----------|-------------------|-------------|------------------------|
| default   | hello-app-service |             | http://127.0.0.1:53322 |
|-----------|-------------------|-------------|------------------------|
* Abrindo o serviço default/hello-app-service no navegador padrão...
! Como você está usando um driver Docker no windows, o terminal precisa estar aberto para executá-lo.
```

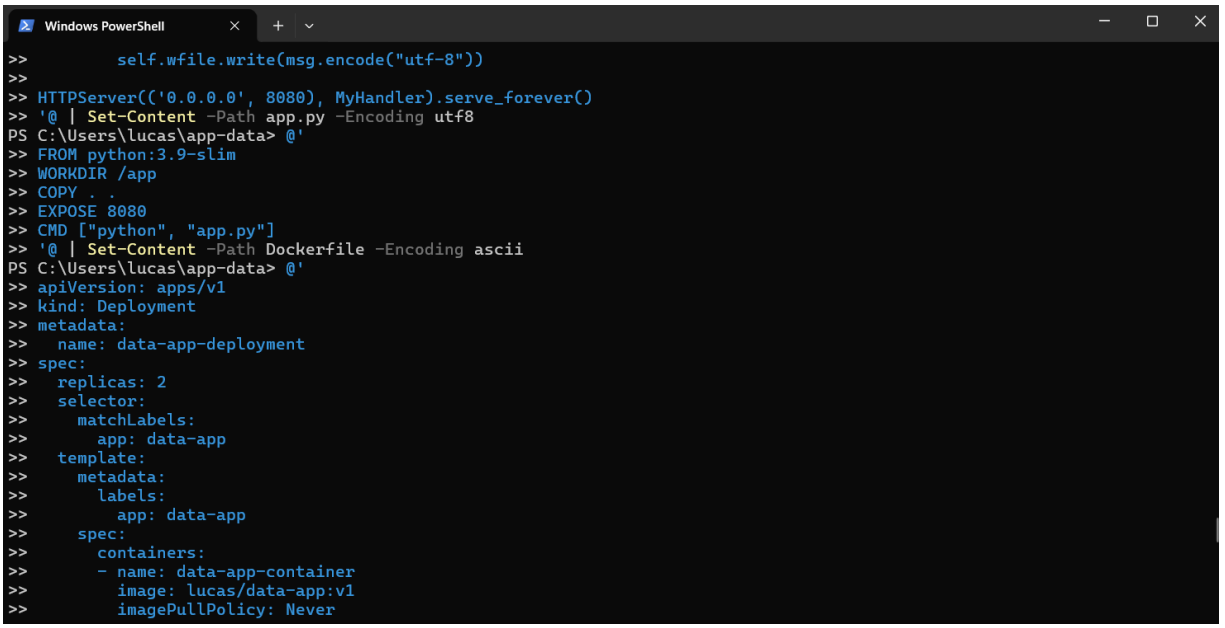


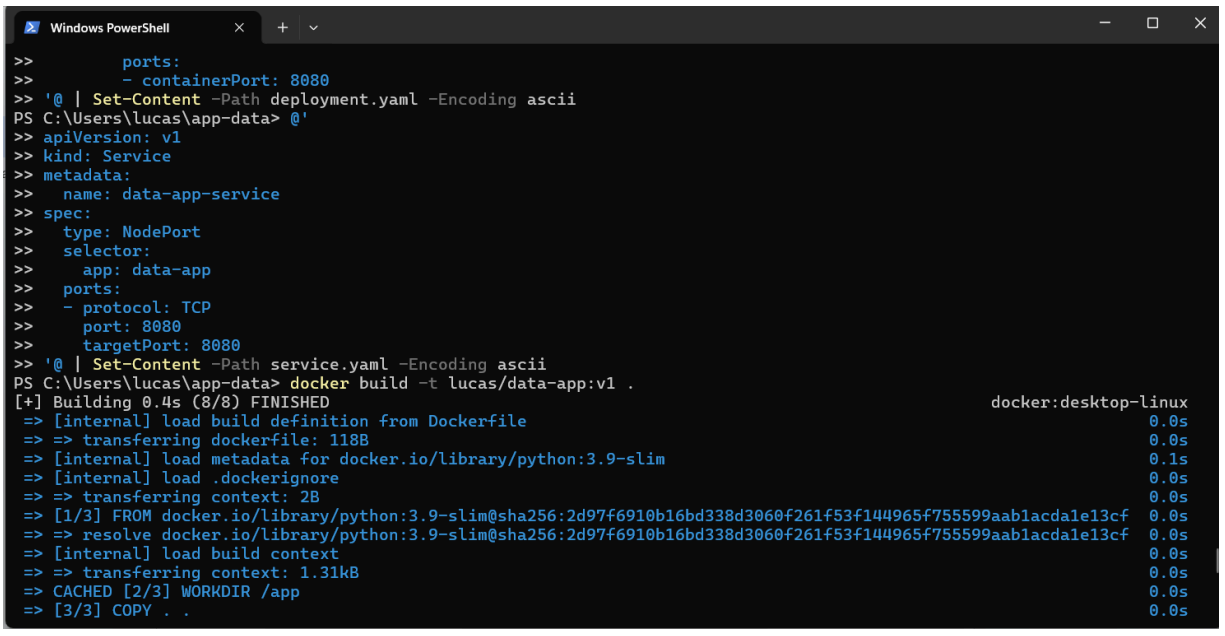


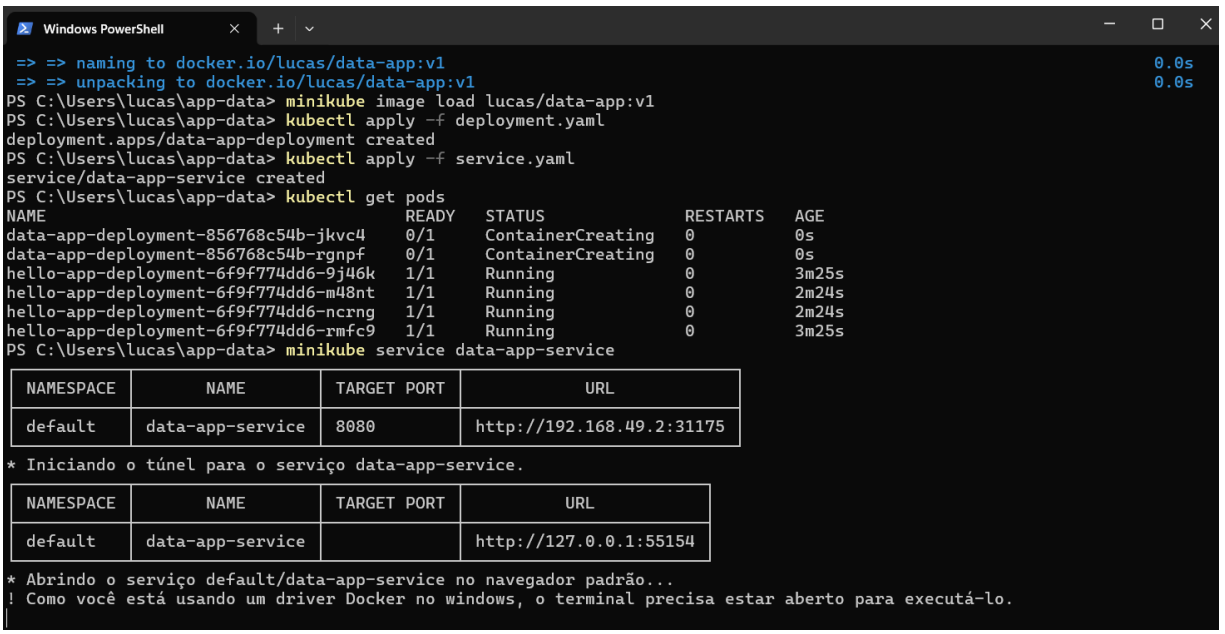


Aluno: Lucas - Data atual: 15/10/2026 03:06:34


Windows PowerShell
Diretório: C:\Users\lucas
Mode LastWriteTime Length Name
d----- 15/10/2026 00:05 app-data
PS C:\Users\lucas\docker-k8s> cd C:\Users\lucas\app-data
PS C:\Users\lucas\app-data> @'
>> from http.server import SimpleHTTPRequestHandler, HTTPServer
>> from datetime import datetime
>>
>> class MyHandler(SimpleHTTPRequestHandler):
>> def do_GET(self):
>> agora = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
>> msg = f"Aluno: Lucas - Data atual: {agora}"
>> self.send_response(200)
>> self.send_header("Content-type", "text/html; charset=utf-8")
>> self.end_headers()
>> self.wfile.write(msg.encode("utf-8"))
>>
>> HTTPServer(('0.0.0.0', 8080), MyHandler).serve_forever()
>> '@ | Set-Content -Path app.py -Encoding utf8
PS C:\Users\lucas\app-data> @'
>> FROM python:3.9-slim
>> WORKDIR /app
>> COPY . .
>> EXPOSE 8080


Windows PowerShell
>> self.wfile.write(msg.encode("utf-8"))
>>
>> HTTPServer(('0.0.0.0', 8080), MyHandler).serve_forever()
>> '@ | Set-Content -Path app.py -Encoding utf8
PS C:\Users\lucas\app-data> @'
>> FROM python:3.9-slim
>> WORKDIR /app
>> COPY . .
>> EXPOSE 8080
>> CMD ["python", "app.py"]
>> '@ | Set-Content -Path Dockerfile -Encoding ascii
PS C:\Users\lucas\app-data> @'
>> apiVersion: apps/v1
>> kind: Deployment
>> metadata:
>> name: data-app-deployment
>> spec:
>> replicas: 2
>> selector:
>> matchLabels:
>> app: data-app
>> template:
>> metadata:
>> labels:
>> app: data-app
>> spec:
>> containers:
>> - name: data-app-container
>> image: lucas/data-app:v1
>> imagePullPolicy: Never


Windows PowerShell
>> ports:
>> - containerPort: 8080
>> '@ | Set-Content -Path deployment.yaml -Encoding ascii
PS C:\Users\lucas\app-data> @'
>> apiVersion: v1
>> kind: Service
>> metadata:
>> name: data-app-service
>> spec:
>> type: NodePort
>> selector:
>> app: data-app
>> ports:
>> - protocol: TCP
>> port: 8080
>> targetPort: 8080
>> '@ | Set-Content -Path service.yaml -Encoding ascii
PS C:\Users\lucas\app-data> docker build -t lucas/data-app:v1 .
[+] Building 0.4s (8/8) FINISHED docker:desktop-linux
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 118B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 0.1s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/3] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf 0.0s
=> => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 1.31kB 0.0s
=> CACHED [2/3] WORKDIR /app 0.0s
=> [3/3] COPY . . 0.0s


Windows PowerShell
=> => naming to docker.io/lucas/data-app:v1 0.0s
=> => unpacking to docker.io/lucas/data-app:v1 0.0s
PS C:\Users\lucas\app-data> minikube image load lucas/data-app:v1
PS C:\Users\lucas\app-data> kubectl apply -f deployment.yaml
deployment.apps/data-app-deployment created
PS C:\Users\lucas\app-data> kubectl apply -f service.yaml
service/data-app-service created
PS C:\Users\lucas\app-data> kubectl get pods
NAME READY STATUS RESTARTS AGE
data-app-deployment-856768c54b-jkvc4 0/1 ContainerCreating 0 0s
data-app-deployment-856768c54b-rgnpf 0/1 ContainerCreating 0 0s
hello-app-deployment-6f9f774dd6-9j46k 1/1 Running 0 3m25s
hello-app-deployment-6f9f774dd6-m48nt 1/1 Running 0 2m24s
hello-app-deployment-6f9f774dd6-ncrng 1/1 Running 0 2m24s
hello-app-deployment-6f9f774dd6-rmfc9 1/1 Running 0 3m25s
PS C:\Users\lucas\app-data> minikube service data-app-service
NAMESPACE NAME TARGET PORT URL
default data-app-service 8080 http://192.168.49.2:31175
* Iniciando o túnel para o serviço data-app-service.
NAMESPACE NAME TARGET PORT URL
default data-app-service http://127.0.0.1:55154
* Abrindo o serviço default/data-app-service no navegador padrão...
! Como você está usando um driver Docker no windows, o terminal precisa estar aberto para executá-lo.